

Objetivo do Estudo: Construir a arquitetura corporativa de microsserviços em containers Docker/Swarm, com separação estrita entre a camada web (Django + Gunicorn), borda e segurança (Cloudflare + Traefik), mensageria/jobs assíncronos (RabbitMQ + Celery) e agentes autônomos de Inteligência Artificial (LangChain + LangGraph).

1. Equipe Especialista & Papéis Ativos (AGENTS.md)

Subagente	Especialidade Principal	Atribuições no Projeto
Arquiteto de Soluções arquiteto_solucoes	Desenho Sistêmico & Governança	Garantir coerência técnica, contratos de APIs, modelos de dados e avaliação de trade-offs.
Engenheiro DevOps & Infra engenheiro_devops	Docker, Swarm, Traefik & Linux	Estruturar Dockerfiles multi-stage, compose stacks, roteamento dinâmico no Traefik e hardening.
Engenheiro Backend & Filas engenheiro_backend	Django, Redis & RabbitMQ	Settings modulares, pool PostgreSQL, mensageria AMQP robusta e Celery Workers/Beat.
Engenheiro de IA engenheiro_ia	IA Generativa, LangGraph & RAG	Grafos de decisão cognitivos desacoplados da requisição web, pipelines de IA e pgvector.

2. Matriz de Serviços e Portas da Infraestrutura

Serviço / Container	Imagem Base	Portas Internas	Porta Exposta	Papel Arquitetural
Traefik v3.1	traefik:v3.1	80, 443, 8080	80, 443 (HTTP/S)	Proxy Reverso Ingress, Let's Encrypt e auto-discovery
Django + Gunicorn	python:3.12-slim	8000	Nenhuma (via Traefik)	Aplicação web, APIs REST, admin e regras de negócio
PostgreSQL 16	postgres:16-alpine	5432	Nenhuma (rede interna)	Banco relacional transacional ACID (suporta pgvector)
Redis 7	redis:7-alpine	6379	Nenhuma (rede interna)	Cache de alta velocidade, sessões e Celery Results
RabbitMQ 3	rabbitmq:3-management	5672, 15672	15672 (VPN/Admin)	Broker de mensageria AMQP para filas resilientes
Celery Worker	python:3.12-slim	-	Nenhuma	Processamento em segundo plano e execução de IA
Celery Beat	python:3.12-slim	-	Nenhuma	Agendador periódico de rotinas (crontab interno)

3. Roteiro Passo a Passo & Tasklist de Acompanhamento

Fase 0: Fundação de Documentação e Governança Inicial

STATUS: CONCLUÍDO (100%)

- ✓ **Diretório Base:** Criação de estudo_arquitetura_sistemas_pycoder no padrão corporativo.
- ✓ **Documentos Base:** Elaboração de README.md, ARQUITETURA.md, AGENTS.md e MEMORIA.md.
- ✓ **Diagrama Oficial:** Alocação do diagrama de referência em docs/arquitetura_deploy_scsi.png.

Fase 1: Harness do Antigravity 2.0 & Ambiente Local (Passos 1 e 2)

arquiteto_solucoes / engenheiro_backend

- ☐ **Passo 1.1 - Diagnóstico do Host:** Checar Python 3.12+, Docker Desktop, WSL2, Git e uv.
- ☐ **Passo 1.2 - Runtimes & Ferramental:** Criar ambiente virtual e instalar linter ruff e pytest.
- ☐ **Passo 1.3 - Arquivos de Governança:** Criar .editorconfig, pyproject.toml e template .env.example.
- ☐ **Passo 2.1 - Git Local:** Configurar .gitignore corporativo (bloquear secrets e caches) e git init.
- ☐ **Passo 2.2 - GitHub Remoto:** Conectar repositório via SSH e estruturar CI básica com GitHub Actions.

Fase 2: Borda, Domínio & Criptografia (Passos 3 e 4)

engenheiro_devops

- ☐ **Passo 3.1 - Cloudflare DNS:** Configurar registros A/CNAME apontando para o IP da VPS com Proxy Laranja ativo.
- ☐ **Passo 3.2 - Segurança Perimetral:** Configurar WAF, mitigação DDoS e regras de cabeçalho de cache.
- ☐ **Passo 4.1 - Criptografia SSL/TLS:** Configurar modo **Full (Strict)** no Cloudflare.
- ☐ **Passo 4.2 - Certificados de Origem:** Gerar Cloudflare Origin CA ou configurar desafio ACME no Traefik.

Fase 3: Infraestrutura de Servidor & Hardening (Passos 5 e 6)

engenheiro_devops

- ☐ **Passo 5.1 - Provisionamento Hostinger:** Selecionar VPS (mínimo 4 vCPUs / 8-16GB RAM) e mapear firewall.
- ☐ **Passo 6.1 - Usuário & SSH:** Criar usuário sudo, configurar chaves Ed25519 e desabilitar login root.
- ☐ **Passo 6.2 - Firewall UFW & SWAP:** Bloquear portas externas exceto 22, 80 e 443; configurar 4-8GB de SWAP.
- ☐ **Passo 6.3 - Segurança do SO:** Instalar e configurar fail2ban contra ataques de força bruta.

Fase 4: Orquestração & Ingress Dinâmico (Passos 7 e 8)

engenheiro_devops

- ☐ **Passo 7.1 - Docker & Swarm:** Instalar Docker Engine oficial e executar docker swarm init.
- ☐ **Passo 7.2 - Redes Overlay:** Criar redes isoladas traefik-public e internal-net.
- ☐ **Passo 8.1 - Setup Traefik:** Montar traefik.yml com entrypoints 80, 443 e redirecionamento HTTPS.
- ☐ **Passo 8.2 - Dashboard & SSL:** Habilitar dashboard administrativo protegido com BasicAuth e SSL ativo.

Fase 5: Core da Aplicação & Camada de Persistência (Passos 9, 10 e 11)

engenheiro_backend

- ☐ **Passo 9.1 - Django Boilerplate:** Configurar settings modulares (base.py, dev.py, prod.py).
- ☐ **Passo 9.2 - Dockerfile Multi-stage:** Imagem enxuta de produção rodando com Gunicorn e WhiteNoise.
- ☐ **Passo 10.1 - Redis Cache:** Configurar container Redis para cache de alta performance e sessões Django.
- ☐ **Passo 11.1 - PostgreSQL 16:** Subir Postgres com volume persistente e suporte à extensão pgvector.
- ☐ **Passo 11.2 - Migrações & Backup:** Testar migrações do Django e rotina de backup com pg_dump.

Fase 6: Mensageria Assíncrona & Background Jobs (Passos 12, 13 e 14)

engenheiro_backend / engenheiro_devops

- ☐ **Passo 12.1 - RabbitMQ Broker:** Subir RabbitMQ, criar vhosts, credenciais de produção e filas duráveis.
- ☐ **Passo 13.1 - Celery Worker:** Criar celery.py, configurar filas AMQP e Result Backend no Redis.
- ☐ **Passo 13.2 - Resiliência:** Implementar políticas de retry com backoff exponencial para falhas temporárias.
- ☐ **Passo 14.1 - Celery Beat:** Configurar agendamento de tarefas periódicas (*crontab* interno do sistema).

Fase 7: Inteligência Artificial & Agentes Cognitivos (Passo 15)

engenheiro_ia

- ☐ **Passo 15.1 - Provedores de IA:** Gestão segura de chaves de API (OpenAI, Gemini, Anthropic) via .env.
- ☐ **Passo 15.2 - Grafos no LangGraph:** Implementar fluxos de decisão com nós, estado persistido e checagens.
- ☐ **Passo 15.3 - Execução Não-Bloqueante:** Invocar agentes cognitivos dentro dos Celery Workers.
- ☐ **Passo 15.4 - Entrega ao Usuário:** Notificação do status da IA para a interface Django (Polling / HTMX).

4. Diretrizes de Operação do Harness Antigravity 2.0

Ciclo de Feedback Automatizado:

- Toda escrita de código Python é validada com `ruff check --fix`.
- Validações de sintaxe e testes unitários executados via `pytest`.
- Containers validados com healthchecks antes de cada aprovação.

Governança de Memória & Rastreabilidade:

- Atualização do `MEMORIA.md` a cada decisão técnica (ADR).
- Transparência de comandos executados e logs de erros.
- Segurança absoluta: zero commit de arquivos `.env` ou senhas.